Object-Oriented Programming with PHP (Abstracts and Interfaces)

Abstraction

Classes, defined as abstract, cannot have instances (cannot create object of this class)

- Abstract class must have at least one abstract method
- Abstract methods do not have implementation (body) in the class
  - Only signature
- The class must be inherited
- The child class must implement all abstract methods

Abstraction Example

```
abstract class AbstractClass {
        abstract protected function getValue();
        abstract public function getValue2($prefix);

        public function printOut () {
                echo $this->getValue();
        }
}
class Class1 extends AbstractClass {
        protected function getValue (){
                return "Class1";
        }
        public function getValue2($prefix) {
                return $prefix."NAC1";
        }
}
```

Abstraction Example (2)

```
// continue from previous slide
class Class2 extends AbstractClass {
        protected function getValue (){
                return "Class2";
        }
        public function getValue2($prefix) {
                return $prefix."NAC2";
        }
}

$class1 = new Class1();
$class1->printOut(); // "Class1";
echo $class1->getValue2('FOO'); // FOONAC1

$class2 = new Class2();
$class2->printOut(); // "Class2";
echo $class2->getValue2('FOO'); //FOONAC2
```

Interfaces

Object interfaces allow you to specify what methods a child class must implement

- Declared with the `interface` keyword
- Similar to abstract class
- Interface can have only public methods

Interface Example

```
interface iTemplate {
  public function set ($name, $value);
  public function getHTML($template);
}
class Template implements iTemplate {
  private $vars = array();
  public function set ($name, $value) {
    $this->vars[$name] = $value;
  }
  public function getHTML($template) {
   foreach($this->vars as $name=>$value) {
      $template = str_replace(
         '{'.$name.'}', $value, $template);
        }
    return $template;
  }
}
```

End of Topic